

Building REST API with AWS Lambda and Amazon API Gateway

Table of Contents

Introduction	2
Tasks	2
Architectural Overview	3
Prerequisites	3
Implementation	4
Create a Lambda function	4
To create the function	4
Add environment variables	5
Create a REST API using API Gateway	6
To create the API	6
Integrate the API HTTP methods	8
Create a DynamoDB table	12
To create the DynamoDB table	12
Test the integration of API Gateway, Lambda, and DynamoDB	13
Deploy the API	14
To deploy the API	14
Import an API into Postman	15
Test API with Postman	16
Clean up your resources (optional)	19
To delete the Lambda function	19
To delete the API	19
To delete the DynamoDB table	19

Introduction

A RESTful API (Representational State Transfer API) is a specific architectural style for designing application programming interfaces (APIs). It uses standard HTTP methods to access and manipulate data:

- **GET:** Retrieve data
- **PUT:** Update existing data
- **POST:** Create new data
- **DELETE:** Remove data

This design provides a straightforward, predictable way to work with the resources (data) exposed by the API.

In AWS, [Amazon API Gateway](#) is used to manage RESTful APIs. Amazon API Gateway is a fully managed service that simplifies the process of creating, publishing, maintaining, monitoring, and securing APIs at any scale. With Amazon API Gateway, developers can build RESTful APIs for real-time, two-way communication applications.

Key capabilities of Amazon API Gateway include:

- High-performance API requests and responses
- Authorization through AWS IAM and Amazon Cognito, with native OAuth support
- Simultaneous versioning to iterate and test new API versions
- Monitoring of API performance metrics, usage, and error rates

Tasks

This article will guide you through the process of creating a RESTful API that triggers a Lambda function using an HTTP request. The Lambda function will handle create, read, update, and delete (CRUD) operations on a DynamoDB table.

To complete the activity in this article, you will go through the following steps:

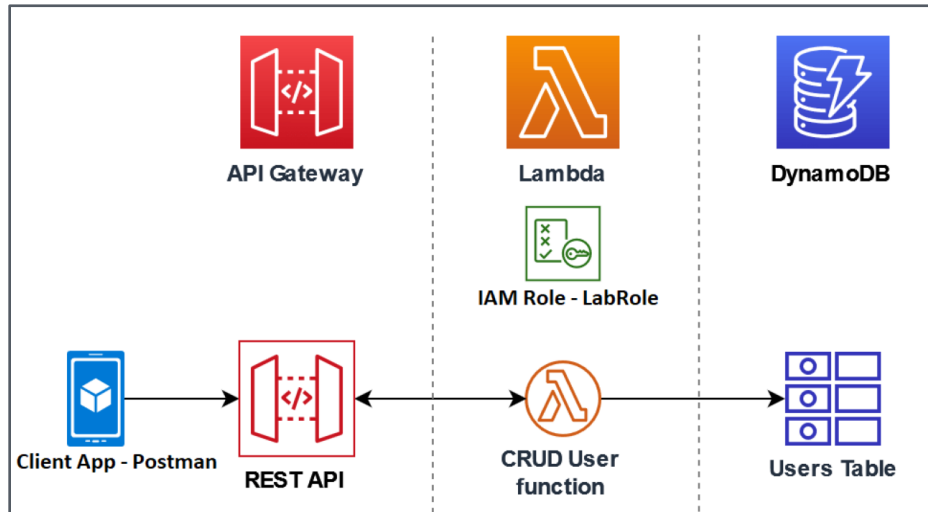
1. Create and configure a Lambda function in Python to perform CRUD (create, read, update, delete) operations on a DynamoDB table.
2. Create a RESTful API in API Gateway that connects to your Lambda function.
3. Create a DynamoDB table and test the Lambda function's interactions with the table using the AWS console.
4. Deploy your API, export it as a Swagger file, and test the full setup using Postman.

The step-by-step instructions for each of these stages will be provided in the article. By the end, you will have a fully functional RESTful API that leverages [AWS Lambda](#) and [Amazon DynamoDB](#) to perform CRUD operations.

AMAZON CONFIDENTIAL



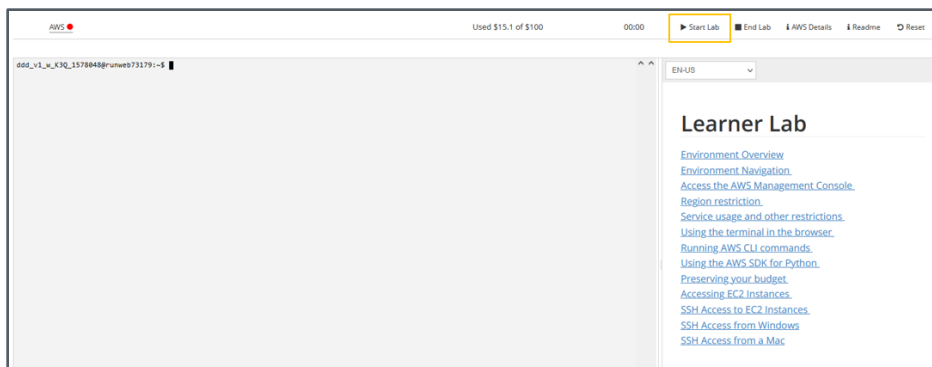
Architectural Overview



Prerequisites

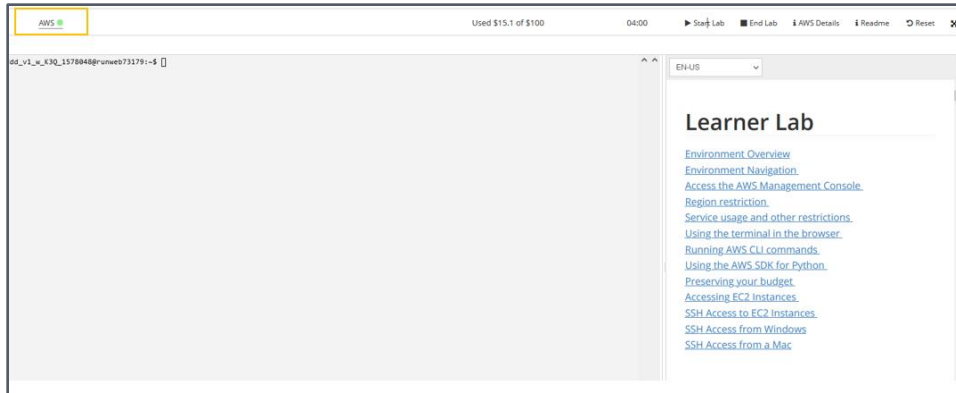
Before continuing with the tasks listed above, you will need:

- **Access to an [AWS Academy Learner Lab \(LL\)](#) course:** From the Student View of the LL, click the Start Lab button as shown in the screenshot below:



Note: For information on how to view a Learner Lab course in student view, please review **How to view a Learner Lab in Student View** section from [AWS Academy Learner Lab Educator Guide](#).

- After starting the lab, wait until the AWS icon turns green as shown below:



Implementation

Create a Lambda function

Before creating a Lambda function, you need to ensure that you have an execution role with the necessary permissions policy to access the required AWS resources. The role must have a policy that allows Lambda to perform CRUD operations on a DynamoDB table and write to Amazon CloudWatch Logs.

Since you are using the AWS Academy Learner Lab for this activity, there is no need to create your own role. The pre-provisioned IAM role, **LabRole**, has the necessary permissions policy to function as your Lambda execution role.

Later in the article, you will need the Amazon Resource Name (ARN) of the LabRole. To obtain the ARN:

1. Navigate to the Roles page in the IAM console.
2. Locate the LabRole and select it.
3. On the Summary page for the LabRole, you will find the Role ARN. Copy this ARN, as you will need it for the next steps in the article.

To create the function

To create your Lambda function, you will use the [CloudShell](#) as your terminal to execute the CLI command. First, you need to download the **Main.zip** file available in the folder accompanying this article. Then, upload the Main.zip file to your CloudShell terminal. Next, use the AWS CLI `create-function` command to create the Lambda function. For the `--role` parameter, enter the Amazon Resource Name (ARN) of the LabRole that you copied earlier.

```
aws lambda create-function \
  --function-name lambda-for-books-api \
```

AMAZON CONFIDENTIAL

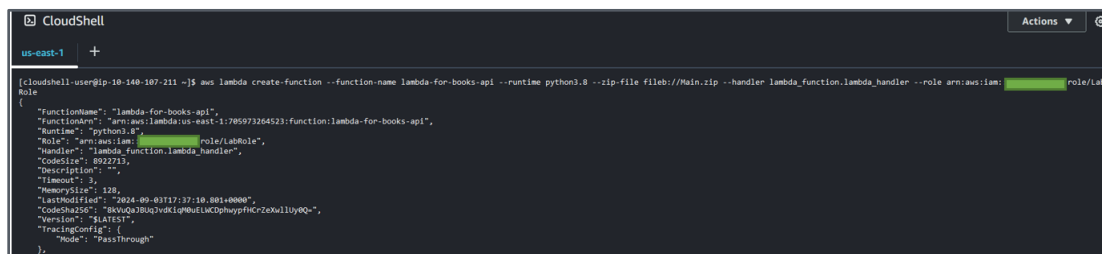


```
--runtime python3.8 --zip-file fileb://Main.zip \
--handler lambda_function.lambda_handler \
--role arn:aws:iam::123456789012:role/role/LabRole
```

The Main.zip file is a deployment package that contains two key components:

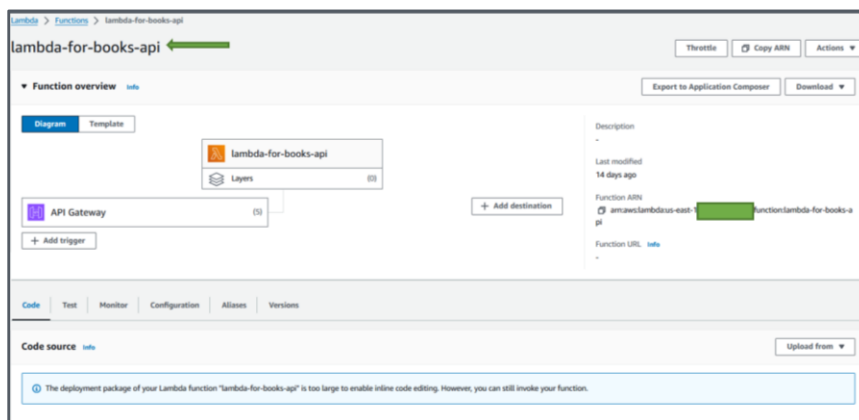
1. The .py file that includes the handler code for your function. This is the code that will be executed when the Lambda function is triggered.
2. Any additional packages or libraries that your function's code depends on. These supporting files are included in the deployment package alongside the main handler code.

The Main.zip file serves as a self-contained package that includes everything needed to deploy and run your Lambda function.



```
CloudShell
us-east-1 +
[cloudshell-user@ip-10-140-107-211 ~]$ aws lambda create-function --function-name lambda-for-books-api --runtime python3.8 --zip-file fileb://Main.zip --handler lambda_function.lambda_handler --role arn:aws:iam::[redacted]:role/LabRole
{
  "FunctionName": "lambda-for-books-api",
  "FunctionArn": "arn:aws:lambda:us-east-1:785973264523:function:lambda-for-books-api",
  "Runtime": "python3.8",
  "Role": "arn:aws:iam::[redacted]:role/LabRole",
  "Handler": "lambda_function.lambda_handler",
  "CodeSize": 8922713,
  "Description": "",
  "Timeout": 3,
  "MemorySize": 128,
  "LastModified": "2024-09-03T17:37:10.001-0000",
  "CodeSha256": "88vUqz78Uq7dKlqHbULWdPhyppfHC7exulloyQ=",
  "Version": "14AT51",
  "TracingConfig": {
    "Mode": "PassThrough"
  },
}
```

After successfully executing the AWS CLI command to create the Lambda function, navigate to the AWS Lambda console. Your new function should now be visible and should appear similar to the following:



Add environment variables

To complete this step, you need to set up environment variables. Environment variables are key-value pairs that are stored in a function's configuration.

AMAZON CONFIDENTIAL

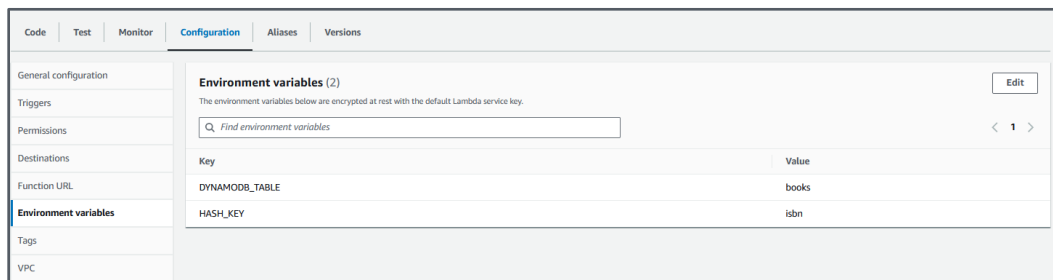


The first environment variable is the name of the DynamoDB table, and the second one is the hash key for that table. You can use these environment variables to adjust your function's behavior without updating the code. For example, if you need your function to work with a different DynamoDB table, you can simply change the environment variables instead of rewriting your Lambda code.

To set these two environment variables on your function, execute the following CLI command:

```
aws lambda update-function-configuration \
--function-name lambda-for-books-api \
--environment "Variables={DYNAMODB_TABLE=books,HASH_KEY=isbn}"
```

After successfully executing the AWS CLI command to update your Lambda function, navigate to the AWS Lambda console. Select the **Configuration** tab, then choose **Environment variables** from the left navigation menu. The new environment variables that you configured should be visible and appear similar to the following:



NB – If the new environment variables you configured don't immediately appear in the Lambda console, you may need to refresh the function page to see the updated environment variable settings.

Create a REST API using API Gateway

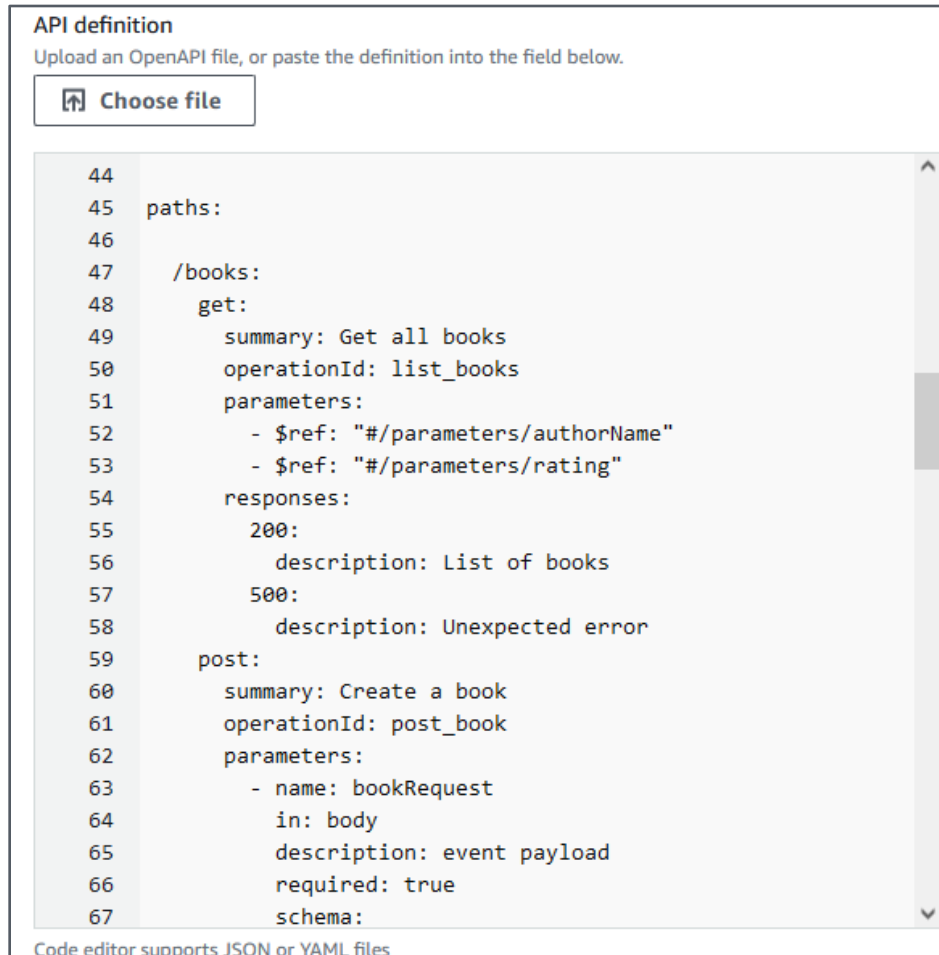
In this step, you create the API Gateway REST API you will use to invoke your Lambda function.

To create the API

1. Open the [API Gateway console](#).
2. In the **REST API** type box, choose **Import**.
3. In the **API definition** section, select **Choose file** and navigate to the location of the **swagger.yaml** file, and upload the file. The swagger.yaml file is one of the files provided in the folder accompanying this article.
4. Choose **Create API**.

The Swagger file is written in YAML, which is a human-readable data serialization format. The **swagger.yaml** file is your API specification. It contains general information about the API, as well as the technical details of every endpoint exposed by the API.

Here's an example snippet from the Swagger file that shows the definition of an API endpoint:



In the Swagger file snippet provided, an API endpoint with the root path **/books** is defined. This endpoint accepts HTTP **GET** requests and can return two possible responses:

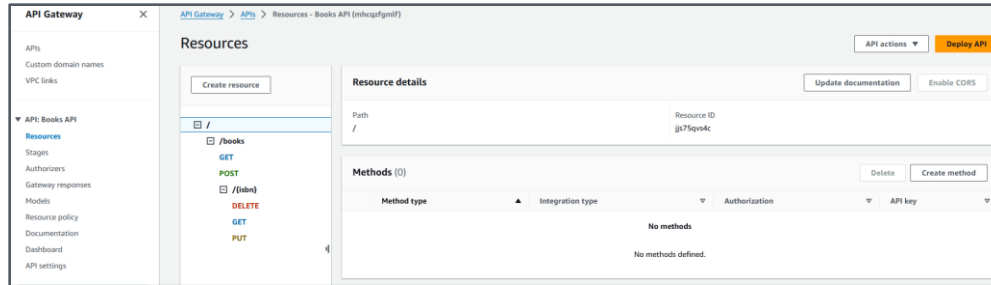
1. HTTP status code 200, which indicates a successful response and returns a list of books.
2. HTTP status code 500, which indicates an unexpected error.

The technical details of the endpoint, such as the request method, response status codes, and response payloads, are all specified in this Swagger file snippet.

Once you have successfully created your API, you will see the various methods that were defined in the Swagger file being generated and available for use.

AMAZON CONFIDENTIAL





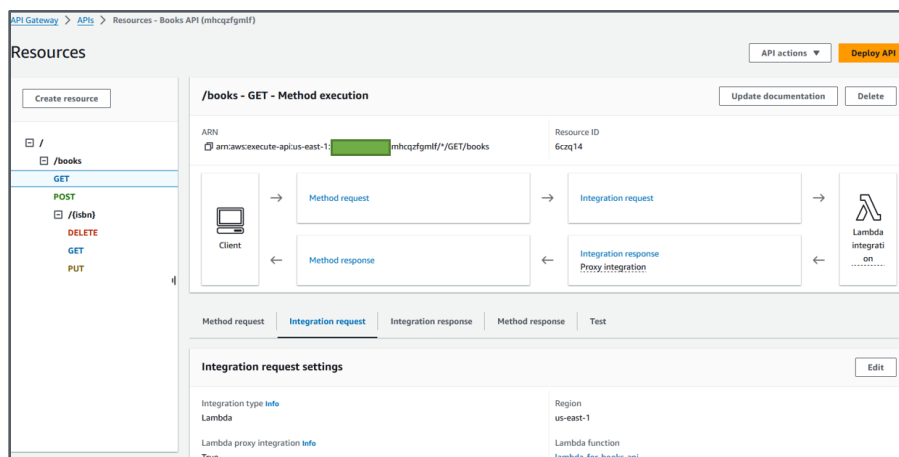
Integrate the API HTTP methods

In this step, you link the HTTP methods to your Lambda function so that when the method receives an HTTP request, API Gateway invokes your Lambda function.

Define GET Method

1. On the **Resources** page for your API, ensure the `/books` resource is selected. Then, click the **GET** method.
2. On the **GET Method** execution page, click the **Edit integration** button.
3. For the **Integration type**, leave **Lambda function** selected. Then, toggle the **Lambda proxy integration** option to the right.
4. For the **Lambda function**, enter the Amazon Resource Name (ARN) for your function (lambda-for-books-api).
5. For the **Execution role**, enter the Amazon Resource Name (ARN) of the **LabRole**.
6. Click **Save** to apply the changes.

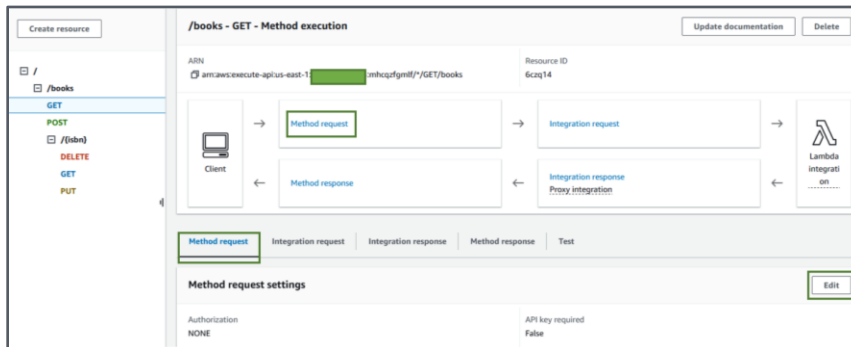
After successfully saving your GET method setup, you will be redirected to the **Method execution** page. This page will display the flow of your GET request, as shown in the example below:



Since you enabled proxy integration, no transformation is applied at the API level. To ensure your **requests, headers, and query parameters** are validated by API Gateway, you will need to enable the request validator for all the HTTP methods.

Enable a request validator on your GET method

1. On the **Method execution** page (shown in the previous screenshot), locate the **Method request** section. Click the **Edit** button to make changes to the method request.



2. In the **Request validator** section, select the option to **validate the request body, query string parameters, and headers**.

The screenshot shows the 'Edit method request' dialog box. The 'Method request settings' section is expanded. The 'Authorization' dropdown is set to 'None'. The 'Request validator' dropdown is highlighted with a green box and shows the option 'Validate body, query string parameters, and headers'. Below this, there is a checkbox for 'API key required' which is unchecked. The 'Operation name - optional' field contains the text 'list_books'. At the bottom, there are three expandable sections: 'URL query string parameters', 'HTTP request headers', and 'Request body'. The 'Cancel' and 'Save' buttons are at the bottom right.

- Review the other settings on the **Method request** page, then click **Save** to apply the changes.

The screenshot shows a configuration page for a method request. It is divided into three main sections:

- URL query string parameters:** This section contains a table with columns for Name, Required, and Caching. It lists two parameters: 'authorName' and 'rating', both with 'Required' and 'Caching' checkboxes set to false. Each parameter has a 'Remove' button next to it. Below the table is an 'Add query string' button.
- HTTP request headers:** This section shows 'No headers found' and an 'Add header' button.
- Request body:** This section shows 'No models found' and an 'Add model' button.

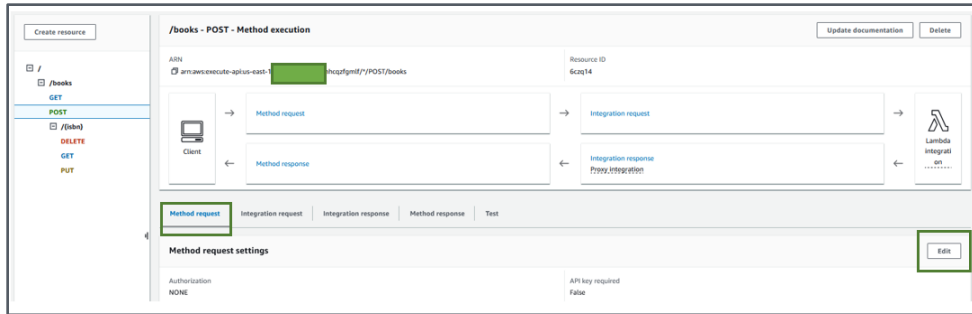
At the bottom right of the form, there are 'Cancel' and 'Save' buttons.

Define POST method

- On the **Resources** page for your API, ensure the `/books` resource is selected. Then, click the **GET** method.
- On the **POST Method** execution page, click the **Edit integration** button.
- For the **Integration type**, leave **Lambda function** selected. Then, toggle the **Lambda proxy integration** option to the right.
- For the **Lambda function**, enter the Amazon Resource Name (ARN) for your function (`lambda-for-books-api`).
- For the **Execution role**, enter the Amazon Resource Name (ARN) of the **LabRole**.
- Click **Save** to apply the changes.

Enable a request validator on your POST method

- On the **Method execution** page, locate the **Method request** section. Click the **Edit** button to make changes to the method request.



2. In the **Request validator** section, select the option to In the **Request validator** section, select the option to **validate the request body, query string parameters, and headers**.

The screenshot shows the 'Edit method request' form. The 'Method request settings' section includes a dropdown for 'Authorization' set to 'None', a dropdown for 'Request validator' set to 'Validate body, query string parameters, and headers' (highlighted with a green box), an unchecked checkbox for 'API key required', and a text input for 'Operation name - optional' with the value 'post_book'. Below this are three expandable sections: 'URL query string parameters', 'HTTP request headers', and 'Request body'. At the bottom right are 'Cancel' and 'Save' buttons.

3. Review the other settings on the **Method request** page, then click **Save** to apply the changes.

▼ URL query string parameters

No query strings found

Add query string

▼ HTTP request headers

No headers found

Add header

▼ Request body

Content type: application/json

Model: postBookRequest

Remove

Add model

Cancel Save

Repeat the steps for defining the GET or POST method for the **DELETE**, **GET**, and **PUT** methods under the **/isbn** resource. Additionally, follow the instructions for enabling a request validator on these methods to activate all three.

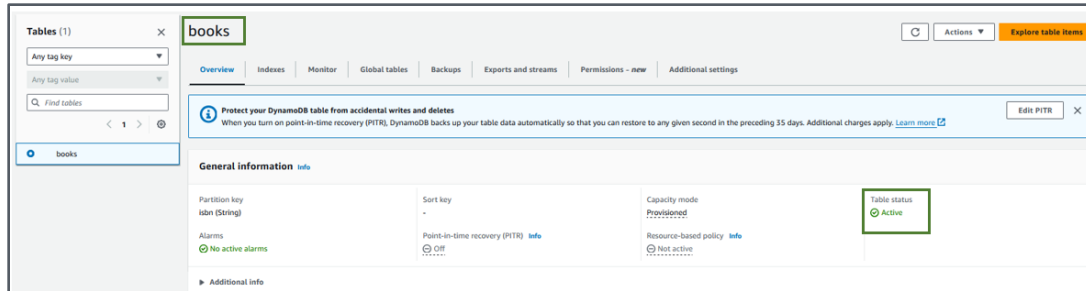
After defining and enabling request validators for all methods, you can review the configurations by clicking on the **Gateway responses**, **Models**, **Documentation**, and **API settings** menus of your Books API, as provided by the Swagger file you imported to create it.

Create a DynamoDB table

Create an empty DynamoDB table that your Lambda function will perform CRUD operations on.

To create the DynamoDB table

1. Open the [Tables page](#) of the DynamoDB console.
2. Choose **Create table**.
3. Under **Table details**, do the following:
 - a. For **Table name**, enter *books*.
 - b. For **Partition key**, enter *isbn*, and keep the data type set as String.
4. Under **Table settings**, keep the **Default settings**.
5. Choose **Create table**.

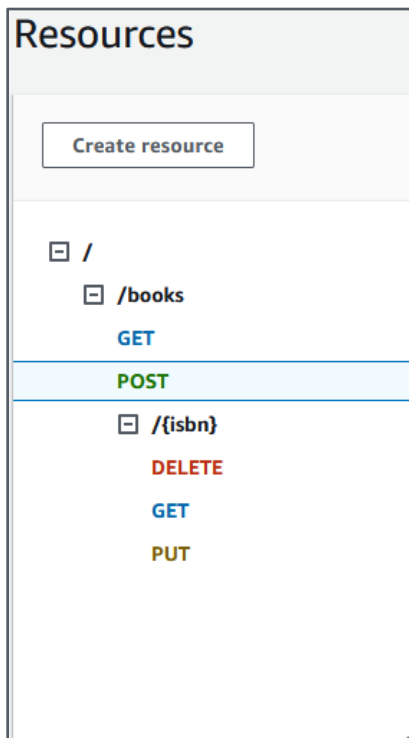


Test the integration of API Gateway, Lambda, and DynamoDB

You are now ready to test the integration of your API Gateway method with your Lambda function and DynamoDB table. Using the API Gateway console, you can send requests directly to your **POST** method through the console's test function. In this step, you'll first perform a create operation to add a new item to your DynamoDB table. Once the API is deployed, you'll test the **read**, **update**, and **delete** operations to fully verify the functionality of your API.

Create a new item in your DynamoDB table

1. In the [API Gateway console](#), choose your API (*BooksAPI*).
2. Choose the **POST** method under the */books* resource.



3. Choose the **Test** tab.

- Under **Test method**, leave **Query strings** and **Headers** empty. For **Request body**, specify the following JSON to include the four required attributes:

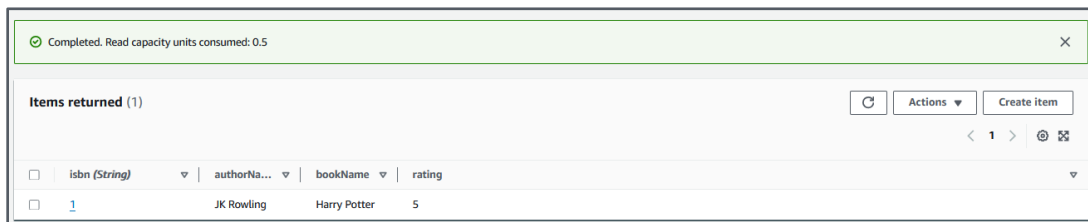
```
{
  "isbn": "1",
  "bookName": "Harry Potter",
  "authorName": "JK Rowling",
  "rating": 5
}
```

- Choose **Test**.

The results that are displayed when the test completes should show status 200. This status code indicates that the create operation was successful. To confirm, check that your DynamoDB table now contains the new item.



- Open the [Tables page](#) of the DynamoDB console and choose the **books** table.
- Choose **Explore table items**. In the **Items returned** pane, you should see one item with these details: `{\"isbn\": \"1\", \"bookName\": \"Harry Potter\", \"authorName\": \"JK Rowling\", \"rating\": 5}`



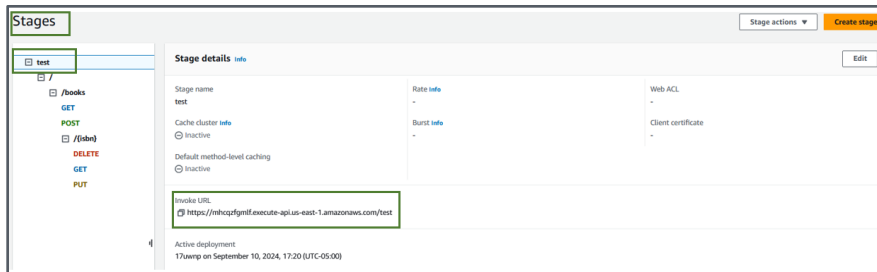
Deploy the API

For a client to call the API, you must create a deployment and an associated stage. A stage represents a snapshot of your API including its methods and integrations.

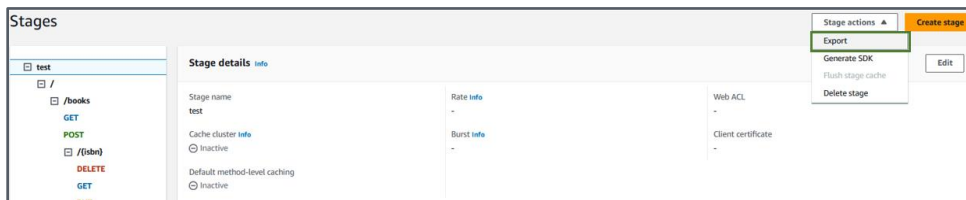
To deploy the API

- Open the APIs page of the [API Gateway console](#) and choose the **Books API**.

2. On the **Resources** page for your API choose **Deploy API**.
3. For **Stage**, choose ***New stage***, then for Stage name, enter **test**.
4. Choose **Deploy**.



5. From the **Stage actions** dropdown menu, select **Export**. On the **Export API** page, choose the option to **Export with Postman extensions**. You will use the exported Swagger file in the next step to invoke your function using an HTTP requests.



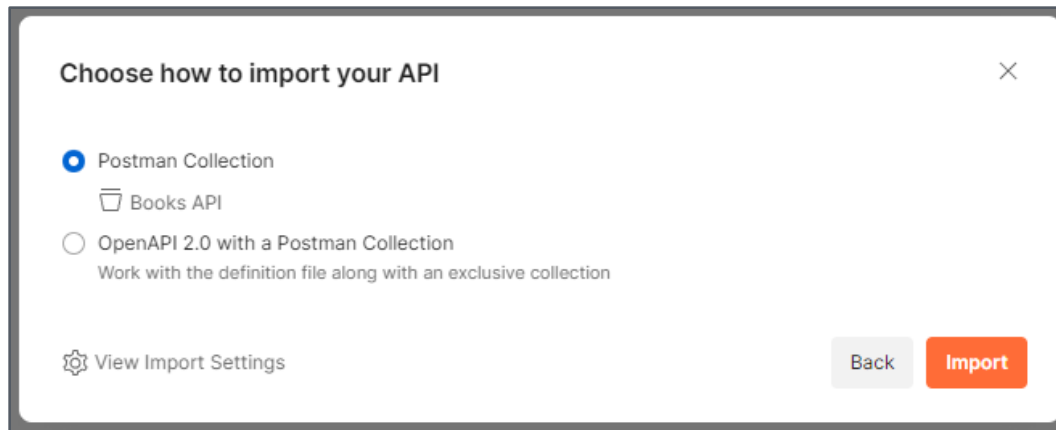
Import an API into Postman

Postman is a powerful API platform that simplifies the entire API lifecycle and enhances collaboration, allowing teams to create better APIs more efficiently. Its user-friendly interface, comprehensive testing tools, and real-time collaboration features set it apart from other API clients. Additionally, Postman's version control, rich ecosystem of integrations, and strong community support further accelerate API development and improve overall quality.

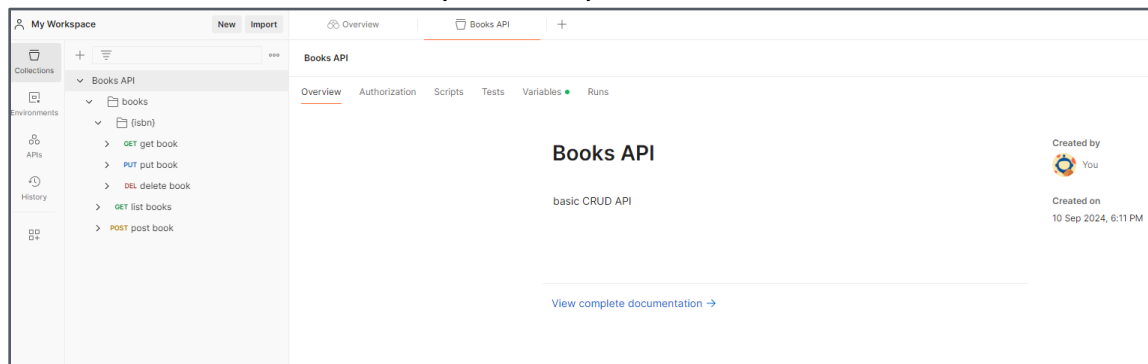
If you are new to using Postman, review the [Get started in Postman](#) documentation. This will provide guidance on how to download and install the Postman desktop application, as well as how to navigate the Postman user interface.

Import an API definition

1. In Postman, select **Import** in the sidebar.
2. Select the API Swagger file that you downloaded earlier.
3. You can choose to import the definition as a collection or as an API along with a collection.



4. Select **Import**.
5. An **Import Complete** message displays in the footer. In the message, select the link icon  next to a collection or API to open the imported element.



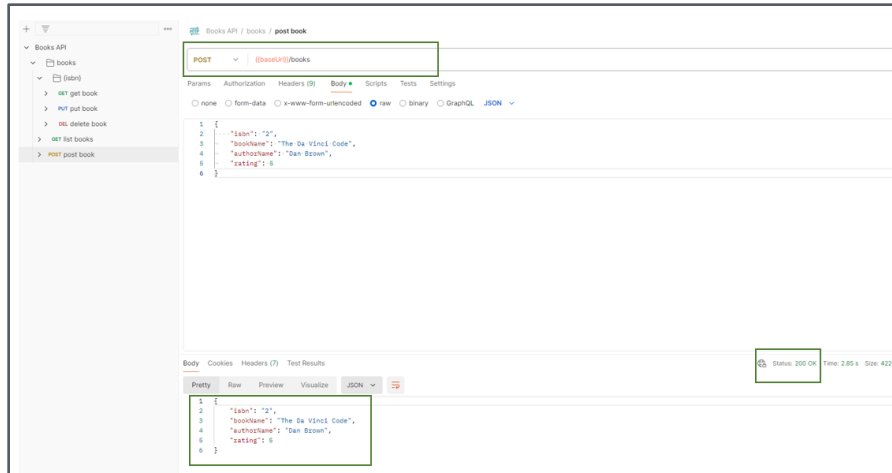
Test API with Postman

The Postman API client enables you to send API requests using HTTP or other common protocols.

Send a POST Request

1. To create a new **POST** request, click on the POST method in your imported Books API. Select the **Body** tab, then choose **JSON** from the available request format options.
2. In the Request body, update the values of the four required attributes to values different from those you specified earlier.
3. Choose **Send**.

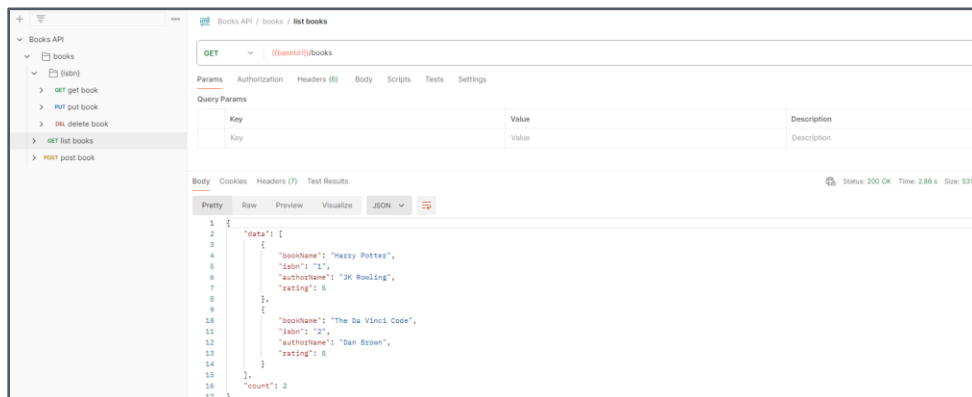
A successful POST request should look like the following:



Send a GET all Requests

1. To retrieve all items (list books), click on the **GET** method for **list books** in your imported Books API.
2. Modify the URL to end with **/books**.
3. Click **Send**.

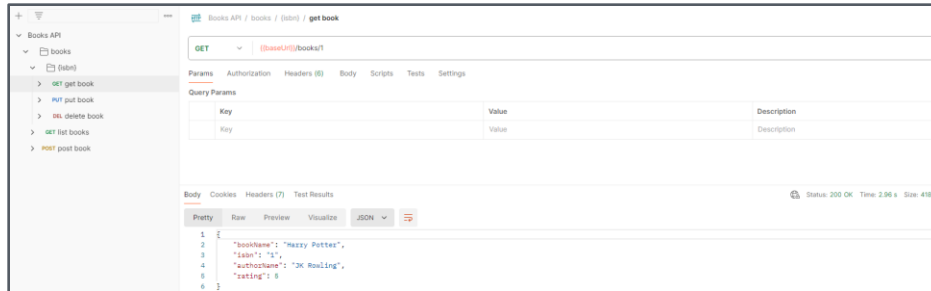
A successful GET all books request should look like the following:



Send a GET a Request

1. To retrieve a book (get book), click on the **GET** method for **get book** in your imported Books API.
2. Replace `:isbn` in the URL with the specific ISBN of the book you want to retrieve.
3. Click **Send**.

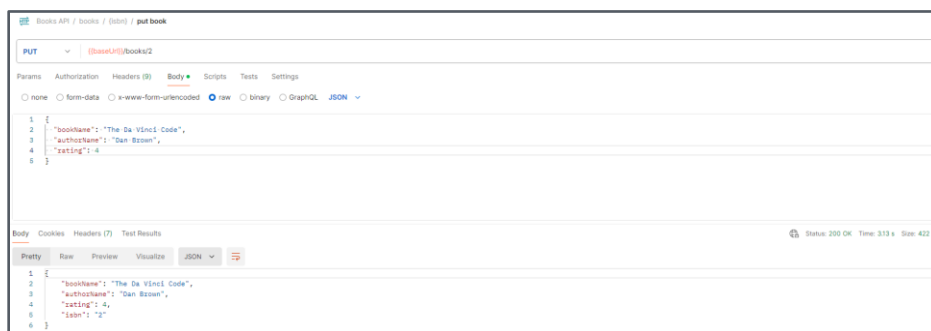
A successful GET request should look like the following:



Send a PUT Request

1. To update a book, click on the **PUT** method in your imported Books API. Select the **Body** tab and choose **JSON** from the request format options.
2. In the Request body, modify the values of any of the three attributes, and replace **:isbn** with the specific ISBN of the book you want to update.
3. Click **Send**.

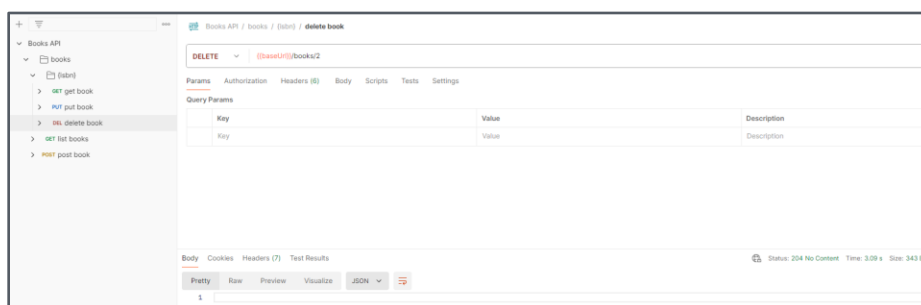
A successful PUT request should look like the following:



Send a DELETE Request

1. To delete a book, click on the **DELETE** method in your imported Books API.
2. Replace **:isbn** in the URL with the specific ISBN of the book you want to delete.
3. Click **Send**.

A successful DELETE request should look like the following:



AMAZON CONFIDENTIAL



NB: A successful status code for a DELETE request is 204, indicating no content.

Clean up your resources (optional)

You can now delete the resources that you created for this tutorial, unless you want to retain them. By deleting AWS resources that you're no longer using, you prevent unnecessary charges to your AWS account.

To delete the Lambda function

1. Open the [Functions page](#) of the Lambda console.
2. Select the function that you created.
3. Choose **Actions, Delete**.
4. Type delete in the text input field and choose **Delete**.

To delete the API

1. Open the [APIs page](#)
1. of the API Gateway console.
2. Select the API you created.
3. Choose **Actions, Delete**.
4. Choose **Delete**.

To delete the DynamoDB table

1. Open the [Tables page](#) of the DynamoDB console.
2. Select the table you created.
3. Choose **Delete**.
4. Enter delete in the text box.
5. Choose **Delete table**.